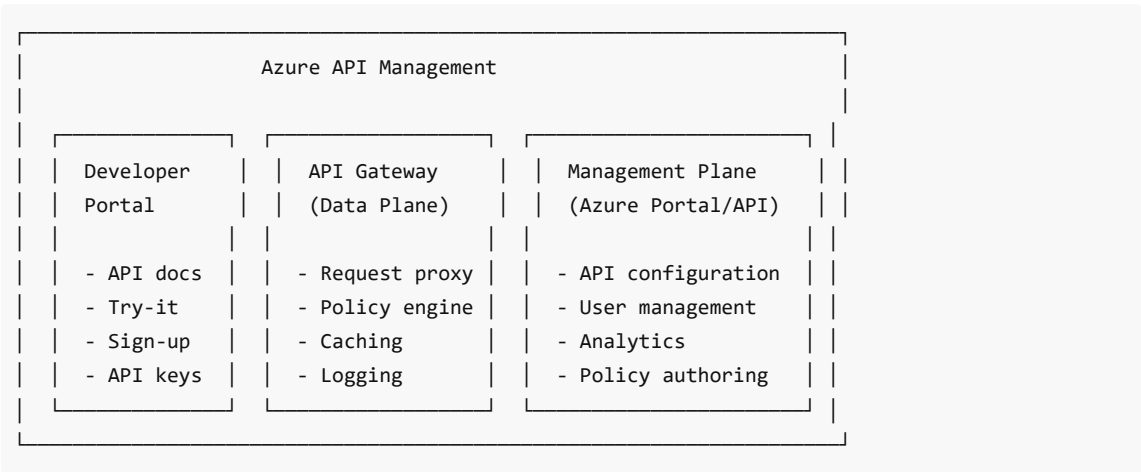


Azure API Management — End-to-End Architecture

Overview

Azure API Management (APIM) is a fully managed service that enables organizations to publish, secure, transform, maintain, and monitor APIs. It acts as a **reverse proxy** (API Gateway) sitting between API consumers and backend services.

Core Components



1. API Gateway (Data Plane)

The gateway is where all API calls flow through. It:

- Receives incoming HTTP/HTTPS requests
- Applies **inbound policies** (authentication, rate limiting, transformation)
- Forwards requests to the **backend service**
- Applies **outbound policies** (response transformation, header manipulation)
- Returns the response to the caller

2. Management Plane

The Azure Portal (or REST API / Bicep / Terraform) is used to:

- Define APIs (OpenAPI, WSDL, GraphQL)
- Configure policies
- Manage products, subscriptions, and users
- View analytics and diagnostics

3. Developer Portal

A customizable website where API consumers can:

- Discover and explore APIs
 - Generate subscription keys
 - Test API calls interactively
-

End-to-End Request Flow

```
sequenceDiagram
    participant Client as 🖥️ API Consumer
    participant WAF as 🛡️ Azure WAF / App Gateway
    participant APIM as 🌐 APIM Gateway
    participant Policy as 📄 Policy Engine
    participant Backend as 💻 Backend API (Azure)
    participant Monitor as 📊 Azure Monitor

    Client->>WAF: 1. HTTPS Request
    WAF->>WAF: 2. WAF Rules (SQL injection, XSS)
    WAF->>APIM: 3. Forward clean request

    rect rgb(173, 216, 230)
        Note over APIM,Policy: INBOUND POLICY PIPELINE
        APIM->>Policy: 4. Execute inbound policies
        Policy->>Policy: ✓ Validate JWT token
        Policy->>Policy: ✓ Check rate limit
        Policy->>Policy: ✓ Validate request schema
        Policy->>Policy: ✓ Check IP allowlist
        Policy->>Policy: ✓ Transform request
    end

    rect rgb(144, 238, 144)
        Note over APIM,Backend: BACKEND POLICY PIPELINE
        APIM->>Backend: 5. Forward to backend
        Backend->>Backend: 6. Process business logic
        Backend->>APIM: 7. Return response
    end

    rect rgb(255, 218, 185)
        Note over APIM,Policy: OUTBOUND POLICY PIPELINE
        APIM->>Policy: 8. Execute outbound policies
        Policy->>Policy: ✓ Strip internal headers
        Policy->>Policy: ✓ Mask sensitive data
        Policy->>Policy: ✓ Add security headers
        Policy->>Policy: ✓ Transform response
    end

    APIM->>Monitor: 9. Log telemetry
    APIM->>Client: 10. Return secured response

    rect rgb(255, 200, 200)
        Note over APIM,Policy: ON-ERROR (if any step fails)
        APIM->>Policy: Handle error gracefully
        Policy->>Policy: ✓ Custom error response
        Policy->>Policy: ✓ Log error details
        APIM->>Client: Return safe error message
    end
```

Step-by-Step Breakdown

Step	What Happens	Where
1	Client sends HTTPS request to API endpoint	Internet → Azure
2	Azure WAF/Application Gateway inspects for L7 attacks (SQLi, XSS, bot patterns)	Edge
3	Clean request forwarded to APIM gateway	Azure network
4	Inbound policies execute: JWT validation, rate limiting, IP checks, schema validation	APIM Gateway
5	Request forwarded to backend (private endpoint, managed identity auth)	APIM → Backend
6	Backend processes business logic	App Service / AKS / Functions
7	Backend returns response	Backend → APIM
8	Outbound policies execute: strip headers, mask PII, add CORS/security headers	APIM Gateway
9	Telemetry logged to Azure Monitor / Application Insights	Monitoring
10	Secured, transformed response returned to client	APIM → Client

Network Architecture

```
graph TB
  subgraph Internet
    Client[👤 API Consumers]
  end

  subgraph Azure["☁ Azure Cloud"]
    subgraph PublicZone["Public Zone"]
      AFD[Azure Front Door / App Gateway + WAF]
    end

    subgraph APIMZone["APIM Subnet (Private)"]
      GW[APIM Gateway]
      DevPortal[Developer Portal]
    end

    subgraph BackendZone["Backend Subnet (Private)"]
      AppSvc[Azure App Service]
      AKS[Azure Kubernetes]
      Func[Azure Functions]
    end
  end
```

```
subgraph DataZone["Data Subnet (Private)"]
  SQL[(Azure SQL)]
  Cosmos[(Cosmos DB)]
  KV[Key Vault]
end

subgraph MonitorZone["Monitoring"]
  AI[Application Insights]
  LA[Log Analytics]
end

Client -->|HTTPS| AFD
AFD -->|Private Link| GW
GW -->|Private Endpoint| AppSvc
GW -->|Private Endpoint| AKS
GW -->|Private Endpoint| Func
AppSvc --> SQL
AppSvc --> Cosmos
GW -->|Managed Identity| KV
GW -->|Telemetry| AI
AI --> LA
```

Key Security Architecture Decisions

- 1. **APIM in Internal Mode:** Gateway only accessible via private IP; public traffic routed through App Gateway/WAF
- 2. **Private Endpoints:** Backend services never exposed publicly; connected via Azure Private Link
- 3. **Managed Identity:** APIM authenticates to Key Vault and backends using managed identity (no stored credentials)
- 4. **Network Segmentation:** Separate subnets for APIM, backends, and data tiers
- 5. **TLS Everywhere:** End-to-end encryption, minimum TLS 1.2

APIM Tiers and Deployment Options

Tier	Use Case	SLA	Gateway Locations
Consumption	Serverless, pay-per-call	99.95%	Single region
Developer	Dev/test (no SLA)	None	Single region
Basic	Entry-level production	99.95%	Single region
Standard	Production workloads	99.95%	Single region
Premium	Enterprise, multi-region	99.99%	Multi-region
v2 Tiers	Next-gen, faster provisioning	99.95%+	Flexible

What Makes APIM the Security Control Plane

APIM is not just a proxy — it's a **programmable security boundary**:

1. **Authentication Gateway:** Validates OAuth 2.0/JWT tokens before requests reach backends
2. **Rate Limiter:** Prevents DDoS and abuse with configurable throttling
3. **Schema Validator:** Rejects malformed requests at the edge
4. **Data Masker:** Strips sensitive data from responses before reaching consumers
5. **Audit Trail:** Every API call logged with caller identity, timing, and response codes
6. **Policy Engine:** Turing-complete policy expressions for custom security logic

Key Insight: By enforcing security at the APIM layer, backends can focus on business logic without reimplementing auth, throttling, or input validation.

Next: [Policy Pipeline Deep-Dive →](#)